

Probabilistic projections of human modification of the terrestrial biosphere from 2020 to 2040

Technical methodology of the forecasting model (production model E2c)

Technical companion to the manuscript of Moncrieff et al.

Prepared 27 September 2026 from repository `spatio-temporal`, branch `main`

Abstract

This document specifies, at implementation level, the model that produces the global Human Modification (HM) forecast and hindcast products. A convolutional LSTM encodes a three-step history of HM and its stressor, socio-economic, climatic and neighbourhood covariates on a ~ 1 km global grid. For each of four horizons (+5, +10, +15 and +20 years) it emits a monotone piecewise-linear quantile function per pixel. The whole predictive distribution is the product; no ensemble, recalibration or post-hoc reshaping follows the network. Training minimises the continuous ranked probability score (CRPS), evaluated in closed form. Skill is measured out of sample with five-fold spatial-block cross-validation over all 184,573,321 scored land pixels. Every number, formula and setting below was taken from the code, the checkpoints, or the logs of the production run; where the implementation departs from what one might assume, the document says so. It is the technical companion to the manuscript, whose Methods describe the approach without implementation detail; terminology follows the manuscript.

Contents

1	Overview	2
2	Input data	3
3	Input data transformations	4
4	Data splitting	5
5	Location encoder	6
6	Trunk model	6
7	Quantile heads	7
8	Loss function	8
9	Model training	9
10	Global prediction	10
11	Model evaluation	12
A	Reproduction	14

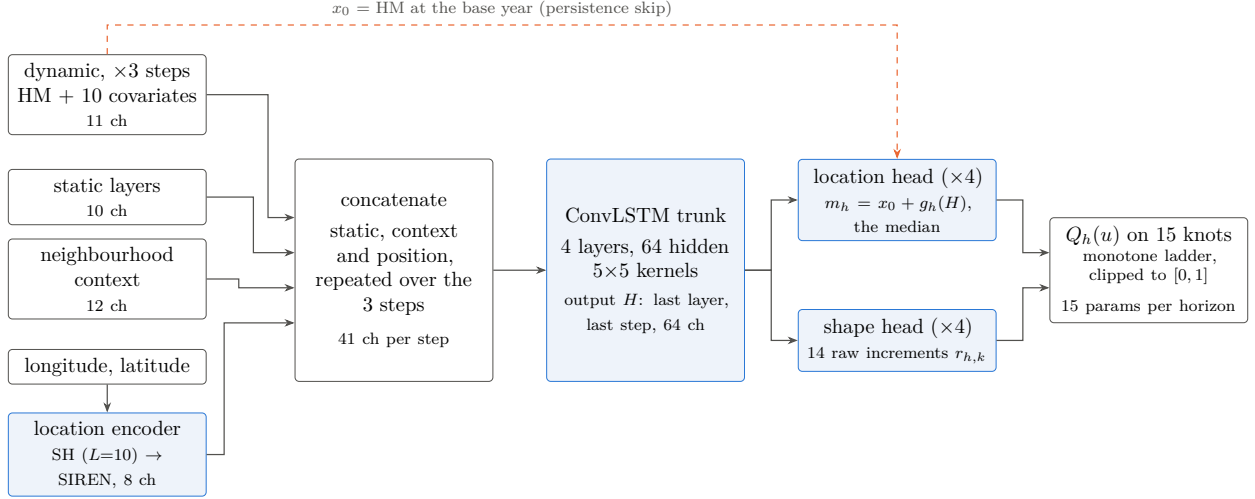


Figure 1: The E2c network. Channel counts are per time step; the ten static channels include slope and aspect. The dashed path adds the base-year HM to the location head’s output, so the untrained model forecasts persistence. Per horizon the output is 1 location + 14 increments = 15 parameters per pixel.

1 Overview

Task. Given HM at three past dates ($t - 10, t - 5, t$) together with covariates, predict the full distribution of HM at $t + 5, t + 10, t + 15$ and $t + 20$ at every land pixel of a global grid. HM quantifies the cumulative impact of 16 anthropogenic threats, grouped into eight thematic classes that map to the IUCN threat taxonomy, as a continuous index on $[0, 1]$. It is available every five years from 1990 to 2020 (Theobald et al., 2025); its change over twenty years is small almost everywhere (the median 20-year change is 0.0001) and heavily right-skewed where it happens.

Products. Two products are built from one model configuration:

- a **hindcast**, base year 2000 (inputs 1990, 1995, 2000; targets 2005, 2010, 2015, 2020). It is a mosaic of five cross-validation models in which every pixel is predicted by the model that never trained on it, so it is out of sample and is what the scores in Section 11 describe;
- a **forecast**, base year 2020 (inputs 2010, 2015, 2020; targets 2025, 2030, 2035, 2040), from a single model trained on all data.

Each is delivered as Cloud-Optimised GeoTIFFs (lower, mean, upper, $P(\text{HM} > 0.1)$, $P(\text{HM} > 0.4)$ per year) and as an icechunk store holding all 64 quantile levels per pixel (Section 10).

The production model, E2c. The network (Figure 1) has three parts.

1. A *location encoder* maps longitude and latitude to 8 features (Section 5).
2. A four-layer *ConvLSTM trunk* with 64 hidden channels and 5×5 kernels reads a 41-channel input at each of three time steps (Section 6).
3. Per horizon, two small convolutional *heads* emit 15 numbers per pixel: a location (the median) and 14 positive increments that define a piecewise-linear quantile function on a fixed grid of 15 quantile levels (Section 7).

The network has 3,365,444 trainable parameters on the production path. It is trained end to end on the CRPS alone (Section 8).

Notation. x_0 is HM at the base year t ; y_h is observed HM at $t + h$; $Q_h(u)$ is the predicted quantile function at horizon $h \in \{5, 10, 15, 20\}$ years and $F_h = Q_h^{-1}$ its CDF; $u \in [0, 1]$ is a quantile

level; μ_{HM} and σ_{HM} are the HM normalisation constants (Section 3). The model works internally in normalised units $z = (\text{HM} - \mu_{\text{HM}})/\sigma_{\text{HM}}$, and everything reported is back-transformed to HM.

2 Input data

Grid. All rasters share one grid: EPSG:4326, 40000×17111 pixels at 0.009° (≈ 1 km at the equator), spanning 180°W – 180°E and 83.997°N – 70.002°S . Of its 684.4 M pixels, 184.6 M are land with valid HM. Nothing is resampled or reprojected at any stage.

Variables. Table 1 lists every input.

- *Response.* Cumulative HM (AA) from Theobald et al. (2025), produced at 300 m and average-resampled to the 1 km grid.
- *HM component threats*, supplied individually as dynamic covariates: agriculture (AG), built environments (BU), extraction (EX), biological resource use (FR), human accessibility (HI), natural systems modification (NS), pollution (PO) and transportation (TI).
- *Socio-economic*, also dynamic: gridded population (Zhuang et al., 2024) and GDP (Murakami et al., 2021), both aligned to the five-year HM years before they reach this repository.
- *Static:* elevation from EarthEnv, derived from GMTED2010 (Amatulli et al., 2018); mean annual temperature (tas), minimum temperature of the coldest month (tasmin) and mean annual precipitation (pr) from CHELSA (Karger et al., 2017); the Development Suitability Index (dpi_dsi) (Oakleaf et al., 2024); and two binary protected-area masks from the WDPA (UNEP-WCMC and IUCN, 2025), strictly protected (IUCN Ia, Ib, II; iucn_strict) and all other categories (iucn_nostrict), applied unchanged to every year; and slope and aspect, derived in this project from the elevation raster (Section 3).

Group	Channels	Files	Content
Target / dynamic	1	HM_{year}_AA_1000.tiff	overall HM, $[0, 1]$; also the first dynamic channel
Dynamic	8	HM_{year}_{v}_1000.tiff	the eight HM component threats AG, BU, EX, FR, HI, NS, PO, TI
Dynamic	2	HM_{year}_gdp/population	gridded GDP (Murakami et al., 2021) and population (Zhuang et al., 2024)
Static	7	hm_static_{v}_1000.tiff	elevation, three CHELSA climatologies, DSI, two WDPA protection masks
Static	3	hm_static_terrain_{v}_1000.tiff	slope, sin and cos of aspect, from the elevation raster
Context	12	change_context_w{t}, hm_context_w{t}	neighbourhood of past change and of current HM (Table 2)
Position	8	(computed)	location-encoder features from longitude and latitude

Table 1: Inputs. Dynamic channels are read at each of the three input dates; the others are read once per sample and repeated across the three steps. Every time step therefore carries $11 + 10 + 12 + 8 = 41$ channels.

Temporal windows. A sample uses input dates $(t - 10, t - 5, t)$ and targets $t + 5, \dots, t + 20$. HM exists only for 1990–2020, so the base year t can be 2000, 2005, 2010 or 2015 during training, and horizons beyond 2020 are missing for the later bases:

base t	inputs	observed targets	usable horizons
2000	1990, 1995, 2000	2005, 2010, 2015, 2020	5, 10, 15, 20
2005	1995, 2000, 2005	2010, 2015, 2020	5, 10, 15
2010	2000, 2005, 2010	2015, 2020	5, 10
2015	2005, 2010, 2015	2020	5

Missing targets are filled with NaN and masked out of the loss. The hindcast is predicted from base 2000 and the forecast from base 2020.

3 Input data transformations

Missing values. The HM rasters declare nodata as the *finite* value 3.4×10^{38} . Every reader uses a masked read, so the sentinel becomes NaN before any arithmetic; an `isfinite` test on the raw values would instead count the ocean as fully modified land. After that:

- the ten dynamic covariates have NaN set to 0 before normalisation, read as “no pressure”;
- static `ele`, `dpi_dsi`, `iucn_nostrict`, `iucn_strict` and the three terrain channels have NaN set to 0 before normalisation; the climate layers keep their NaNs.
- The elevation raster declares no nodata value and fills every row south of 56°S (9.09% of the grid, 558 land pixels) with -32768 . The normalisation file declares that value as a fill, and the reader turns it into NaN before the rule above, so it is read as sea level (0 m) rather than as an elevation. Training and prediction use the same reader.

Normalisation. Every channel except the context is standardised, $\tilde{v} = (v - \mu_v)/\sigma_v$, with per-variable statistics:

- HM (input and target) uses one pooled mean and standard deviation over all seven years;
- each covariate and static layer has its own mean and standard deviation.

The statistics are exact population moments over *land* pixels, computed by one streaming pass over each raster (`scripts/compute_norm_stats.py`). A pixel counts if HM is valid there (finite, non-negative and not the declared nodata) and the channel’s own value is not missing or a declared fill. HM and each dynamic covariate are pooled over all seven years, each year masked by its own HM; the static channels are masked by the 2020 HM. 10^{-8} is added to each standard deviation. The statistics are written to `data/conv_spline/norm_stats_E2c.json`, which also records the elevation fill. They are *not* stored in the checkpoint, so prediction from a checkpoint must use the same file. The production values of the HM constants are $\mu_{\text{HM}} = 0.087103$ and $\sigma_{\text{HM}} = 0.149811$; elevation has $\mu = 675$ m and $\sigma = 845$ m.

Terrain. Slope and aspect are computed once from the elevation raster with the [Horn \(1981\)](#) 3×3 finite-difference gradient (`scripts/prepare_terrain.py`). The grid is geographic, so a pixel is about 1.00 km wide at the equator and 0.12 km at 83°N ; the gradient uses each row’s east–west and north–south pixel size in metres on the WGS84 ellipsoid, and longitude wraps at $\pm 180^\circ$. Slope is in degrees. Aspect, the downslope bearing clockwise from north, is circular, so it enters as `sin` and `cos`, both 0 on flat ground (including the sea, which the elevation raster fills with 0 m). The fill value and every pixel whose 3×3 window touches it are written as NaN. Over land, slope has mean 2.0° and standard deviation 3.4° .

Imputation and validity. Normalisation happens before imputation. Any NaN remaining in an input is then replaced by 0, the variable’s mean in standardised units, so the network sees a finite tensor. A per-pixel validity mask decides which pixels contribute to the loss and to predictions: the target is finite, the HM input is finite at the base year, and every dynamic and static input is finite at every step. Invalid pixels are excluded everywhere downstream.

Neighbourhood context. A 128 px training tile cannot see 100 px of neighbourhood, and the trunk’s receptive field is only 12 px (Section 6). Long-range information therefore enters as covariates precomputed on the full raster for each base year t (Table 2):

- past change $\Delta_{10} = \text{HM}_t - \text{HM}_{t-10}$;
- the Euclidean distance d in pixels to the nearest pixel with $\Delta_{10} > 0.01$ (a distance transform);
- the mean and maximum of HM_t over square windows of side $2r + 1$ for $r \in \{3, 30, 100\}$ px, i.e. 7×7 , 61×61 and 201×201 px. The mean counts valid pixels only, and both are stored as int16×1/32767, with nodata read as 0.

Because they are computed on the full raster, the 30 and 100 px windows are never truncated at a tile edge. Only input-side years are used, so the context carries no information about the targets.

#	Feature	Definition
1–3	occupancy	$\mathbb{I}[d \leq r]$, $r = 3, 30, 100$ px
4	distance	$\log(1 + d)/10$ (d set to 10^4 where undefined)
5	past change	Δ_{10} , raw HM units
6	current HM	x_0 in standardised units
7–9	neighbourhood mean	mean of HM_t within $r = 3, 30, 100$ px
10–12	neighbourhood max	max of HM_t within $r = 3, 30, 100$ px

Table 2: The 12 context channels, read from the rasters for the sample’s base year.

Tiles. Training and validation read 128×128 px tiles. Each sample also carries a per-pixel longitude–latitude grid from the raster transform for the location encoder (Section 5).

4 Data splitting

Spatial blocks. Pixels are grouped rather than split one by one, because HM residuals are spatially correlated. On Africa the residual field’s fitted practical range is 99–166 px, so held-out tiles of one correlation length would sit inside a ring of trained-on neighbours and the held-out skill would be optimistic. The fold mask (`fold_mask_b4_land_1000.tif`) is built in four steps:

1. enumerate the 128 px tiles on a regular lattice that contain at least one land pixel (14,631 tiles, 35.0% of the grid);
2. group them into 4×4 -tile super-blocks of 512×512 px, about four correlation lengths across;
3. shuffle the super-blocks with a fixed seed (42);
4. assign them greedily to whichever of $k = 5$ folds currently holds the fewest tiles.

The folds hold 2,922–2,933 tiles and 36.5–38.1 M land pixels each. Land is HM that is finite, non-negative and not the raster’s declared nodata. Every land pixel of the grid has a fold except 36,262 in the partial tile strip at the grid edge.

Cross-validation round. Fold $f \in \{1, \dots, 5\}$ is predicted by a model for which (Figure 2):

- fold f is held out entirely, never seen in training or validation;
- fold $v = (f \bmod 5) + 1$ is the validation fold;
- the other three folds form the training pool.

A training tile is admitted only if *no* pixel in it belongs to fold f or fold v , and training tiles are not jittered off the lattice, so no training tile overlaps held-out geography. Each training pool holds 8,774–8,786 lattice positions, every one containing land. Validation reads every lattice tile of fold v at stride 1024 px, 42–50 tiles, always with base year 2000.

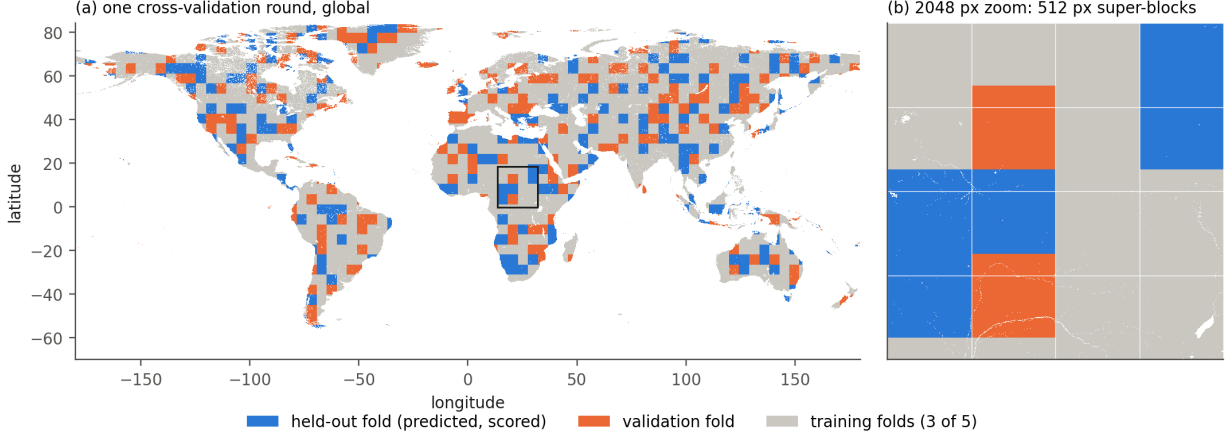


Figure 2: One cross-validation round, drawn on land: fold 1 held out (blue), fold 2 validating (orange), folds 3–5 training (grey). (b) zooms into central Africa, where the white lines are the 512 px super-block boundaries. The five rounds rotate the roles, so every land pixel is held out exactly once.

Forecast model. The forward model holds nothing out: its tiles are drawn uniformly from the whole grid. Its validation set (a fixed 10% split) is therefore in-sample and is only a training monitor. The configuration is justified by the five-fold hindcast, not by the forward model’s own validation.

5 Location encoder

Position enters as learned features of longitude λ and latitude ϕ , computed per pixel, following [Rußwurm et al. \(2023\)](#): spherical-harmonic basis functions passed through a sinusoidal representation network. The encoder lets the model represent residual geographic variation, such as policy regimes or cultural practices, that the covariates do not measure.

Spherical harmonics. With $\varphi = \lambda + 180^\circ$ and $\vartheta = \phi + 90^\circ$ (in radians), the encoder evaluates the real spherical harmonics $Y_\ell^m(\varphi, \vartheta)$ analytically for $\ell = 0, \dots, L - 1$ and $m = -\ell, \dots, \ell$ with $L = 10$ Legendre degrees. That gives a smooth, periodic, pole-consistent basis of $L^2 = 100$ features.

SIREN. The 100 features pass through a sinusoidal network ([Sitzmann et al., 2020](#)) with two hidden layers of 64 units. The hidden layers compute $\sin(\omega_0(Wx + b))$, with $\omega_0 = 30$ in the first layer and 1 after it, and SIREN initialisation. A linear layer then maps to 8 output channels. The encoder has 11,144 parameters. It is trained jointly with the rest of the network by the same optimiser; the learning rate in its own configuration block is not used. Its 8 channels are appended to the static inputs and repeated over the three time steps.

6 Trunk model

Input assembly. At each step $s \in \{1, 2, 3\}$ (dates $t - 10$, $t - 5$, t) the trunk input is the concatenation

$$X_s = \left[\underbrace{\tilde{H}M_s, \tilde{c}_{s,1..10}}_{11 \text{ dynamic}}, \underbrace{\tilde{e}_{1..10}}_{10 \text{ static}}, \underbrace{k_{1..12}}_{12 \text{ context}}, \underbrace{\ell_{1..8}}_{8 \text{ position}} \right] \in \mathbb{R}^{41 \times 128 \times 128},$$

in which the static, context and position blocks are identical at the three steps.

ConvLSTM. Four stacked ConvLSTM layers (Shi et al., 2015), each with 64 hidden channels, 5×5 kernels, padding 2, dilation 1 and bias. For layer l at step s , with input $X_s^{(l)}$ (the data for $l = 1$, the previous layer’s hidden state otherwise),

$$\begin{aligned} [i, f, o, g] &= W^{(l)} * [X_s^{(l)}, h_{s-1}^{(l)}] + b^{(l)}, \\ c_s^{(l)} &= \sigma(f) \odot c_{s-1}^{(l)} + \sigma(i) \odot \tanh(g), \quad h_s^{(l)} = \sigma(o) \odot \tanh(c_s^{(l)}), \end{aligned}$$

where $*$ is a 5×5 convolution producing 4×64 channels and the states start at zero. The trunk output is the last layer’s hidden state at the last step, $H = h_3^{(4)} \in \mathbb{R}^{64 \times 128 \times 128}$. The trunk has 3,130,624 parameters.

Receptive field. Every convolution on the path from an input pixel to H widens the field by the kernel’s half-width, 2px: one per layer and one per recurrent step, and the widest path takes both recurrent steps. The radius is therefore $2 \times (4 + 2) = 12$ px, a 25×25 window. This was checked by back-propagating from a single output pixel. The heads add one more pixel. Everything the model knows about the landscape beyond ± 13 px comes from the context channels and the location encoder, which is why those are precomputed on the full raster.

7 Quantile heads

For each horizon h a separate decoder branch reads H , following the multi-horizon design of Wen et al. (2017). A branch has two heads, each a 3×3 convolution with ReLU followed by a 1×1 convolution. Together they emit a monotone spline quantile function in the sense of Gasthaus et al. (2019), built from non-negative increments as in Park et al. (2022). Here the spline is linear between knots, which is what gives the CRPS a closed form (Section 8).

Location head. $g_h : 64 \rightarrow 64 \rightarrow 1$ channels (36,993 parameters per horizon). Its output is added to the base-year HM, giving the median

$$m_h = x_0 + g_h(H).$$

The output convolution is initialised to zero, so the untrained model forecasts exact persistence and “nothing changes”, the right answer for most of the map, costs nothing to express.

Shape head. $s_h : 64 \rightarrow 32 \rightarrow 14$ channels (18,926 parameters per horizon), emitting raw values $r_{h,1}, \dots, r_{h,14}$. The output weights are initialised to zero and the bias to $\log p_k$, where p_k are the standard-normal bin probabilities of the knot grid. Under the parameterisation below the initial increments are therefore $(|\log p_k| + 10^{-4})c$, which are nearly equal at 0.0028–0.0038 HM per bin: an untrained ladder with a 95% width of about 0.027 HM and a full span of about 0.046 HM.

Knot grid. The quantile function is represented on 15 fixed quantile levels

$$\mathbf{u} = (0, 0.001, 0.005, 0.025, 0.05, 0.10, 0.25, 0.50, 0.75, 0.90, 0.95, 0.975, 0.99, 0.999, 1),$$

denser in the tails, and containing 0.025, 0.5 and 0.975 exactly, so the published interval and median are read directly off knots.

The ladder. In standardised units, the increments are

$$\delta_k = (|r_{h,k}| + 10^{-4})c, \quad c = \frac{w_0}{n_{95}} = \frac{0.065}{8},$$

with $n_{95} = 8$ the number of bins spanning $[0.025, 0.975]$. The constant c is a *unit* (0.00122 HM per unit of $|r|$) that gives the raw outputs a physical scale; it is not a per-pixel normalisation. With cumulative sums $v_0 = 0$, $v_k = \sum_{j \leq k} \delta_j$, the knot values are

$$Q_h(u_k) = \text{clip}\left(m_h + v_k - v_7, \frac{0 - \mu_{\text{HM}}}{\sigma_{\text{HM}}}, \frac{1 - \mu_{\text{HM}}}{\sigma_{\text{HM}}}\right),$$

so the median ($u_7 = 0.5$) is m_h and the knots are clipped to the physical range $\text{HM} \in [0, 1]$. Q_h is the linear interpolant of these knot values. If the 95% span $Q_h(0.975) - Q_h(0.025)$ would fall below 3×10^{-5} HM, all increments are scaled up to reach it, so a zero-width forecast cannot be emitted.

Properties.

- *Monotone by construction.* Every $\delta_k > 0$, so Q_h is non-decreasing and quantiles cannot cross.
- *Free width.* There is no separate scale channel and no coupling across horizons; each horizon's width emerges from its own increments.
- *Bounded support.* Without tail parameters the distribution is supported on $[Q_h(0), Q_h(1)]$. The far tails are carried by the two outermost bins, of probability 0.001 each, rather than by an unbounded extension.
- *Exact derived quantities.* The mean is exact because Q_h is linear on each segment: $\mathbb{E}[Y] = \int_0^1 Q_h(u) du = \sum_k \frac{1}{2} (Q_h(u_k) + Q_h(u_{k+1})) (u_{k+1} - u_k)$. The CDF and the PIT come from inverting the interpolant, and the implied density is piecewise constant, $(u_{k+1} - u_k) / (Q_h(u_{k+1}) - Q_h(u_k))$ (Figure 4).

Outputs. Per pixel and horizon the network therefore emits 15 numbers. From them it derives the triple published beside the quantile function: *lower* = $Q_h(0.025)$, *central* = $\mathbb{E}[Y]$ (the mean, not the median) and *upper* = $Q_h(0.975)$. Because the residual is right-skewed, the published central need not lie at the midpoint of the interval.

Implementation note. The network module also instantiates two lower/upper heads that belong to another output mode of the code (147,976 parameters). They are never called on this path, receive no gradient and keep their initial values; they sit in the checkpoint but play no role in any output.

8 Loss function

Objective. The model minimises the CRPS (Gneiting and Raftery, 2007), written as the integral of the pinball loss over quantile levels:

$$\text{CRPS}(Q, y) = 2 \int_0^1 \rho_u(y - Q(u)) du, \quad \rho_u(e) = e(u - \mathbb{I}[e < 0]).$$

The CRPS is a strictly proper scoring rule, so it trains location, width and shape together. It has bounded influence per pixel, which matters on this data, whose residual kurtosis runs from about 10^3 to 2×10^4 .

Closed form. On a segment $[u_k, u_{k+1}]$ where Q is linear, the error $e(u) = y - Q(u)$ is linear and non-increasing in u . It changes sign at most once, at $u^* = u_k + t^*(u_{k+1} - u_k)$ with $t^* = \text{clip}((y - Q_k)/(Q_{k+1} - Q_k), 0, 1)$. On each side of u^* the integrand $\rho_u(e(u))$ is a quadratic in u , so the segment integral is exact. The implementation writes it as a three-point Simpson evaluation, which is exact for quadratics, and sums the segments:

$$\text{CRPS}(Q, y) = 2 \sum_{k=0}^{13} \left[I_k(0, t_k^*; 0) + I_k(t_k^*, 1; 1) \right],$$

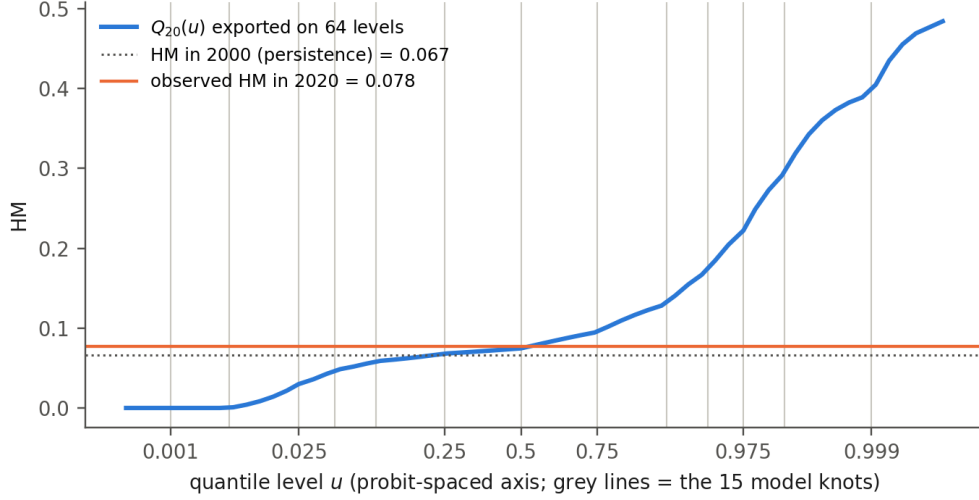


Figure 3: One pixel’s predicted quantile function at +20 years (hindcast, target 2020), as exported on 64 levels. Grey lines mark the model’s 15 knots. The function is linear in u between knots, which appears curved on this probit-spaced axis. Below $u \approx 0.004$ the ladder is clipped at $\text{HM} = 0$. For this pixel $Q(0.025) = 0.030$, median 0.075 and $Q(0.975) = 0.222$, against 0.067 at the base year and 0.078 observed.

where $I_k(a, b; \iota)$ integrates $e(u) (u - \iota)$ over the part of segment k between a and b . No quadrature nodes or approximation are involved. The implementation is checked against a dense numerical reference that shares no code with it.

Reduction. The CRPS is computed in standardised units, and one standardised unit equals σ_{HM} HM. For each horizon it is averaged over valid pixels in the batch (Section 3); a horizon with no valid target in the batch contributes 0. The four horizon means are then averaged with equal weight:

$$\mathcal{L} = \frac{1}{4} \sum_h \frac{\sum_i \mathbb{I}_i \text{CRPS}(Q_{h,i}, y_{h,i})}{\max(1, \sum_i \mathbb{I}_i)}.$$

That is the whole objective. The code’s other terms (MSE on the mean, SSIM, a Laplacian pyramid and a histogram loss) all have weight 0 in production. There is no sample reweighting, because tiles are drawn uniformly, and no weighting across quantile levels.

9 Model training

Sampling. An epoch draws 100 tiles from the training pool, in batches of 8 (13 steps per epoch). For each tile:

- the position is uniform over the admissible lattice positions (hindcast) or over the whole grid (forecast model);
- the base year is drawn uniformly from $\{2000, 2005, 2010, 2015\}$, independently per tile;
- a tile with no finite target is redrawn, up to ten times.

Every admissible hindcast position contains land, so redraws there are rare. The forecast model draws from the whole grid, where about two thirds of tiles contain no land (64.7% of the lattice), so redraws are common and about 1% of its samples (0.647^{10}) still come back empty and contribute nothing to the loss.

Optimisation. The model is implemented in PyTorch (Paszke et al., 2019) with PyTorch Lightning. Adam with learning rate 10^{-3} , $\beta = (0.9, 0.999)$ and no weight decay; constant learning rate;

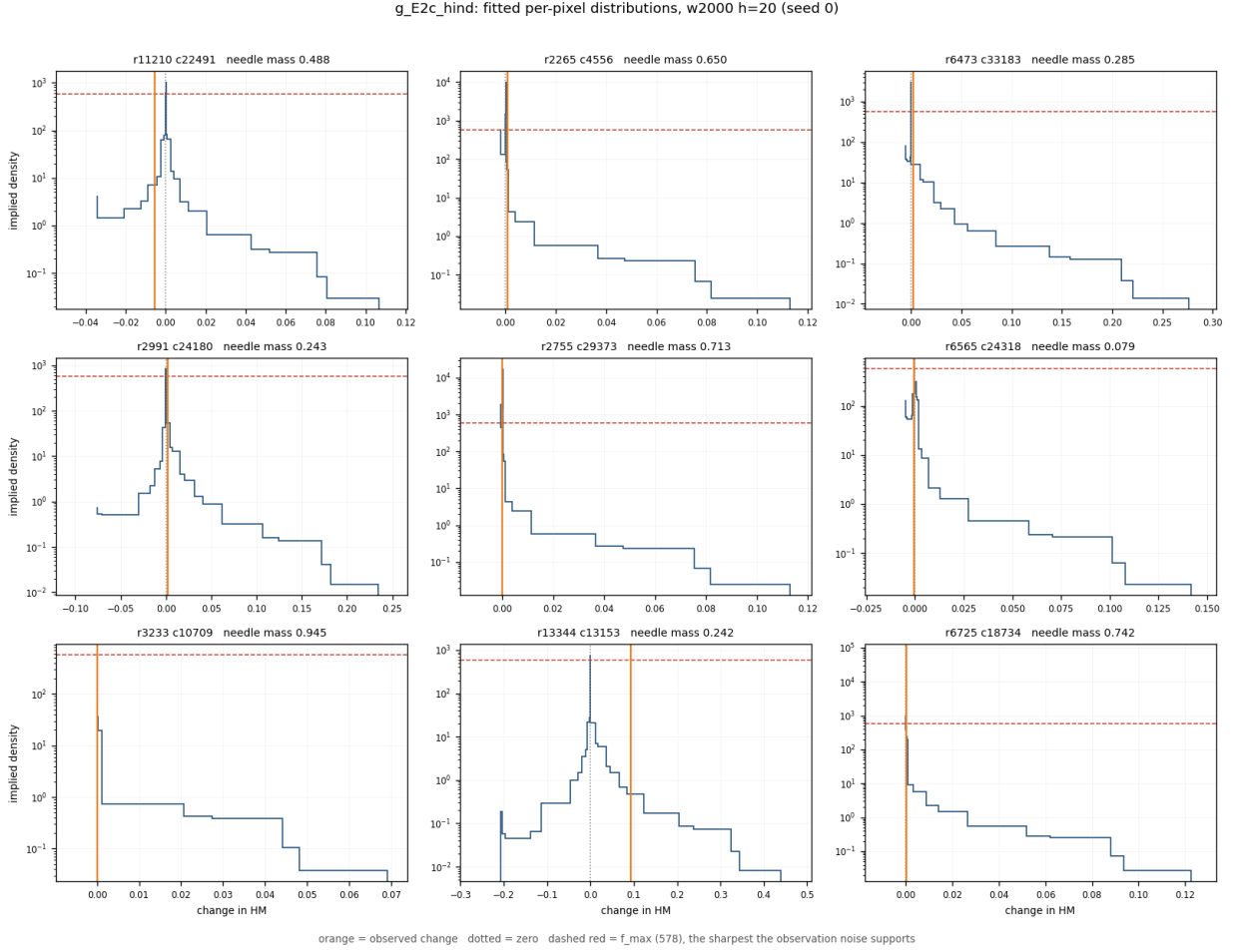


Figure 4: Implied predictive densities of HM *change* at +20 years for nine pixels on a log axis, with the observed change marked in orange. The densities are piecewise constant because Q is piecewise linear. “Needle mass” and the dashed $f_{\max} = 578$ reference are diagnostics for over-sharp segments, discussed in Section 11.

no gradient clipping; float32 precision. The backward pass is manual and end to end: the CRPS gradient reaches the heads, the trunk and the location encoder. The run aborts if any weight becomes non-finite at the end of an epoch. Training lasts 150 epochs, 1,950 optimiser steps.

Validation and the weights used. Each epoch the model is scored on the validation fold’s grid tiles (base year 2000, all four horizons), and the validation CRPS (`val_crps`) is logged and monitored. The weights that predict are *not* the best-validation checkpoint: they are the element-wise mean of the weights at the end of each of the last 20 epochs (131–150), which damps optimisation noise. Held-out folds play no part in any training decision.

Reproducibility and cost. All random number generators are seeded with 42 (Python, NumPy, PyTorch, data-loader workers). On one RTX A5000 (24 GB), training takes about 20 minutes per fold model. A full hindcast fold, training plus global prediction, took 96.2–98.8 minutes with two folds running concurrently on two GPUs. Global prediction is CPU-bound.

10 Global prediction

Tiling and blending. The grid is covered by 128×128 tiles at a stride of 64 px, so every interior pixel is predicted by four tiles. Within a tile, each pixel’s weight is its Euclidean distance to the

Setting	Value	Setting	Value
tile size	128 px	optimiser	Adam, lr 10^{-3} , no decay
batch size	8	schedule / clipping	none / none
tiles per epoch	100 (13 steps)	epochs	150 (1,950 steps)
base years (training)	2000–2015, uniform	weights used	mean of last 20 epochs
validation	fold v , stride 1024, base 2000	monitor	<code>val_crps</code>
trunk	ConvLSTM 4×64 , 5×5	heads	location $64 \rightarrow 64 \rightarrow 1$, shape $64 \rightarrow 32 \rightarrow 14$
location encoder	SH $L = 10$, SIREN 2×64 , 8 out	knots	15 (14 bins)
parameters (used)	3,365,444	seed	42

Table 3: Training configuration of E2c. It is run with `--kernel_size 5 --terrain_covariates True` and its normalisation file (Appendix); every other setting is an argparse default of `scripts/train_lightning.py`.

tile edge (a distance transform, zero on the border and on invalid pixels). Each output quantity is accumulated as $\sum w \hat{q}$ and divided by $\sum w$. Every quantile level is blended with the same weights. A weighted average of non-decreasing sequences is non-decreasing, so the blended result is still a valid quantile function, and because 0.025 and 0.975 lie on the export grid, the blended lower and upper rasters equal the blended quantile levels exactly. Pixels with invalid inputs are set to NaN in every tile. To bound memory, the accumulators are processed in bands of 512 rows. Each band also accumulates every tile that overlaps it, so the result is identical to an unbanded run.

Export grid. The quantile function is written as a 64-band float32 GeoTIFF on a fixed level grid, spaced approximately uniformly in normal score from $u = 2.5 \times 10^{-4}$ to 0.9999, with 0.025, 0.5 and 0.975 pinned as exact levels. Each level is read off the model’s 15-knot function by linear interpolation; the level grid and the head’s family and parameter count are stored as raster tags. The three triple rasters (lower, mean, upper) are written alongside.

Hindcast mosaic. Each fold model predicts every tile that overlaps its held-out fold. The five fold rasters are then stitched in *holdout* mode: each pixel is taken from the fold that held it out. A mean of the five folds would be in sample and is never scored. The stitched rasters cover 184,573,321 land pixels.

Forecast. The forward model predicts the whole grid from base 2020. Its 16 rasters each hold 184,608,551 valid pixels.

Products. Both products are derived from these rasters without any further modelling.

- **COGs**, per year: `lower` = $Q(0.025)$, `mean` = $\mathbb{E}[Y]$, `upper` = $Q(0.975)$ (these three copied bit-exactly), and `gt01` = $1 - F(0.1)$ and `gt04` = $1 - F(0.4)$ (from the scorer’s CDF inversion). These are the probabilities of exceeding the natural-land ($\text{HM} \leq 0.1$) and highly-modified ($\text{HM} > 0.4$) thresholds. Files are named `hm_{year}_{variable}.tif`. Single-band Float32, DEFLATE with floating-point predictor, 7 overview levels, EPSG:4326 at 0.009°.
- **icechunk**: an array `hm(year, percentile, lat, lon)` of shape $4 \times 64 \times 17111 \times 40000$. The values are stored as uint16 codes `min(rint(65535 clip( $q$ , 0, 1)), 65534)`, with scale $1/65535$ and 65535 reserved as the fill value. Chunks are $1 \times 64 \times 256 \times 320$ (10 MB, percentile axis whole) and shards $1 \times 64 \times 1024 \times 1280$.
- **Observed store**: observed HM 1990–2020 on the same grid, encoding and chunk footprint, so observed and predicted tiles align.

Every delivered file is checked against its source rasters, bitwise where the encoding is exact.

Stage (E2c production run)	Wall clock	Peak RAM
hindcast: 5 folds, train + predict, 2 GPUs	4 h 52 min	36 GiB
stitch (16 rasters)	2 h 40 min	36 GiB
forward model: train + global predict	3 h 57 min	36 GiB
score (4 window-years)	5 h 01 min	102 GiB
export, both products in parallel	2 h 52 min	26 GiB each

Table 4: Measured costs on two RTX A5000 GPUs with 125 GB of RAM.

11 Model evaluation

11.1 Protocol and metrics

The stitched hindcast (base 2000, targets 2005–2020) is scored at every land pixel with a finite observation. Each pixel’s prediction comes from the fold model that never trained on it. The reference forecast is **persistence**, $\hat{y} = x_0$. Scoring against zero would be meaningless, since the median change is 0.0001. The metrics:

- **CRPS**, computed exactly for the piecewise-linear function the 64-level raster represents, with the flat tails beyond the outermost levels included in closed form. The CRPS of the point forecast x_0 is $|y - x_0|$, so

$$\text{CRPS skill} = 1 - \frac{\overline{\text{CRPS}}}{\overline{|y - x_0|}}.$$

- **MSE skill** of the mean forecast, $1 - \overline{(\mathbb{E}[Y] - y)^2} / \overline{(x_0 - y)^2}$. The manuscript calls this forecast skill and the scorecards label it “RMSE skill”; it is the MSE ratio in both.
- **Coverage** of the central 50, 80, 95 and 99% intervals, and their mean widths.
- **PIT** $u^* = F(y)$, by inverting the interpolant: its mean (0.5 if centred), the Kolmogorov–Smirnov distance to uniform, and the tail frequencies $P(u^* < 0.001)$ and $P(u^* > 0.999)$ (0.001 each if calibrated). The far-tail excess is $(P(u^* < 0.001) + P(u^* > 0.999))/0.002$, which is 1 when calibrated.
- **Tail reach** $(Q(0.999) - Q(0.5))/(Q(0.975) - Q(0.5))$, and exceedance probabilities $P(\Delta > 0.01)$, $P(\Delta > 0.05)$ and $P(\Delta < -0.01)$ compared with observed frequencies.

Pooled scores average over all pixels, and 99.9% of pixels are nearly static. So every metric is also reported by stratum:

- distance to past change, in bins $[0, 1]$, $(1, 3]$, $(3, 10]$, $(10, 30]$, $(30, 100]$, $(100, \infty)$ px (right-closed);
- predicted change;
- base-year HM;
- observed change.

11.2 Results

The model improves on persistence at every horizon, by 15% in CRPS at +5 years rising to 30% at +15 and +20 years. Skill is carried mostly by pixels near past change: CRPS skill at +20 years falls from 0.42 within 1 px of past change to 0.10 at 30–100 px (Figure 5a).

Horizon	CRPS	CRPS skill	RMSE	MSE skill	cov50	cov95	PIT mean	PIT KS
+5 yr	0.002977	0.155	0.01332	0.041	0.421	0.950	0.519	0.066
+10 yr	0.005318	0.261	0.02033	0.188	0.427	0.917	0.543	0.087
+15 yr	0.007256	0.299	0.02644	0.229	0.449	0.953	0.541	0.087
+20 yr	0.008949	0.299	0.03119	0.241	0.482	0.955	0.540	0.085

Table 5: Out-of-sample hindcast scores of E2c over all 184,573,321 land pixels, base year 2000. CRPS and RMSE are in HM units.

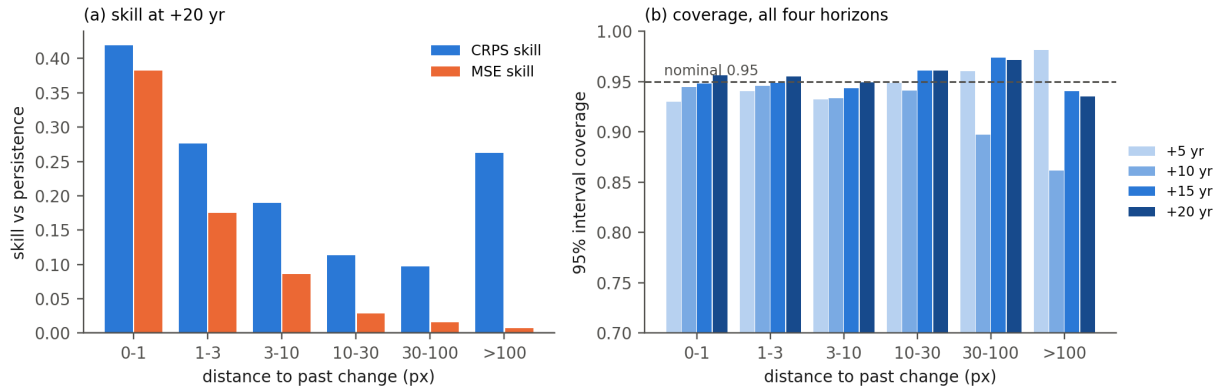


Figure 5: Skill and calibration by distance to past change. (a) CRPS and MSE skill against persistence at +20 years. (b) Coverage of the 95% interval at each horizon; the far field (>100 px, 33.6 M pixels) covers 0.861 at +10 years against 0.95, and 0.94 or better at +15 and +20 years.

Beyond 100 px it rises again, to 0.26, but the point forecast there only ties persistence (MSE skill 0.008). That is a long-horizon result: at +5 years the same band loses slightly to persistence (CRPS skill -0.147 , MSE skill -0.014), and at +10 years it loses badly (-1.445 and -0.166).

Calibration and known limitations.

- **Far-field intervals are too narrow at +10 years.** Beyond 100 px from past change at +10 years, the 95% interval covers 86.1% of observations (and 89.7% at 30–100 px), and the forecast loses to persistence there on CRPS. Pooled over all land this appears as a +10-year coverage of 0.917. At +15 and +20 years the far field covers 0.94 and 0.935. It follows from the bounded support (Section 7) together with widths that are free at each horizon: far-field coverage does not change monotonically with lead time (0.981, 0.861, 0.941 and 0.935 at +5 to +20 years), because nothing constrains how the width grows from one horizon to the next.
- **The far-field point forecast does not beat persistence:** beyond 100 px the MSE skill is -0.014 at +5 years and $+0.008$ at +20 years.
- **The distribution is centred slightly low.** The PIT mean is 0.52–0.54, and the PIT histogram shows recurring spikes near 0.22, 0.52, 0.77 and 0.95 at every horizon (Figure 6).
- **Central intervals are too narrow:** 50% coverage is 0.42–0.48 and 80% coverage 0.76–0.77, while 95% coverage is close to nominal except at +10 years.
- **The lower far tail is over-populated:** $P(u^* < 0.001)$ is 0.0116–0.0125.

The implied densities are very sharp at many pixels (Figure 4). At global scale this is dominated by the physical boundary $HM = 0$, on which most wilderness sits: 71–81% of pixels are flat on a bound, while the fraction with a degenerate segment away from the boundary is 0.000 at every horizon.

g_E2c_hind: PIT, 20 bins

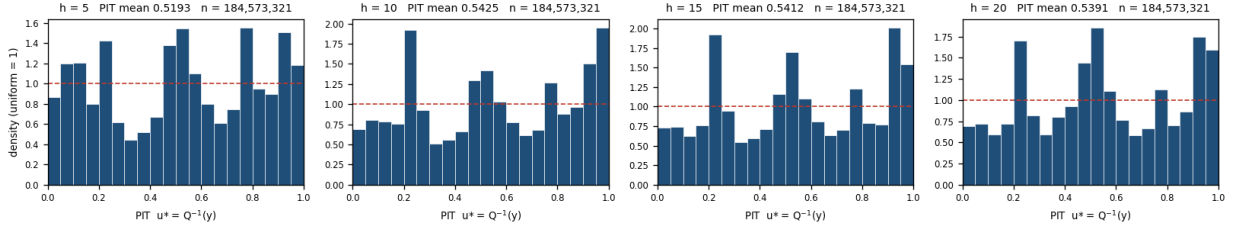
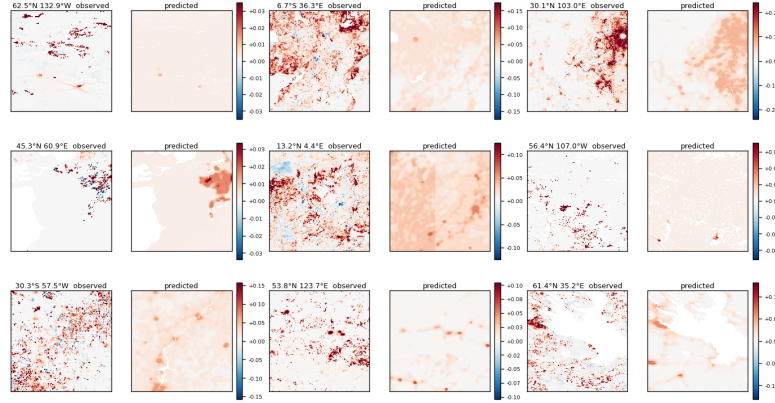


Figure 6: PIT histograms of the hindcast, 20 bins, all land pixels. A calibrated forecast is flat at 1.

Mean HM change 2000→2020: observed vs predicted, nine chips of 256 px



Red = increase, blue = decrease; the scale is shared WITHIN each pair (99th pct of [change]) and differs BETWEEN pairs. Chips are NOT uniform samples: each was required to have more than 2% of its pixels move by over 0.01 HM, because nine uniform draws of a 73%-ocean grid whose land is almost all unchanged would show nothing.

Figure 7: Observed against predicted mean change 2000→2020 for nine 256×256 px tiles, each required to have more than 2% of its pixels change by over 0.01 HM. The predicted field is the mean $\mathbb{E}[Y] - x_0$ and is smoother than the observation, as a mean should be.

A Reproduction

Commands. E2c is not the argparse default configuration of `train_lightning.py`. Its derived inputs are built once with

```
python scripts/prepare_terrain.py
python scripts/compute_norm_stats.py --terrain_covariates
--out data/conv_spline/norm_stats_E2c.json
python scripts/create_validity_mask.py --folds_only --fold_block_chips 4
--fold_min_valid_px 1 --fold_mask_out data/raw/hm_global/fold_mask_b4_land_1000.tif
```

A single E2c model is then trained with

```
python scripts/train_lightning.py --kernel_size 5 --terrain_covariates True
--norm_stats_json data/conv_spline/norm_stats_E2c.json --predict_region ...
```

and a cross-validation fold additionally takes

```
--fold_mask data/raw/hm_global/fold_mask_b4_land_1000.tif.
```

Omitting any of these does not fail; it silently builds a different configuration, which is why the global script below sets them and checks each one in the run's log. The full global run is driven by one script whose stages refuse to start without a passing smoke test of the same code and configuration:

```
MODEL=E2c ALLOW_GLOBAL=1 ./scripts/run_global_model.sh
smoke|hindcast|stitch|score|forecast|export_hindcast|export_forecast
```

Artefacts.

- Checkpoints: `models/production/E2c_global/` (five fold models and the forward model).
- Normalisation statistics: `data/conv_spline/norm_stats_E2c.json`, with a copy beside the checkpoints. Like them it is not versioned in git and must travel with them.
- Scorecards: `docs/global/scores/`.
- Delivered products: `products/E2c/` and `products/observed/` on the project storage volume.

Where things are implemented.

Component	File
data loading, normalisation, sampling	<code>scripts/torchgeo_dataloader.py</code>
normalisation statistics	<code>scripts/compute_norm_stats.py</code>
slope and aspect	<code>scripts/prepare_terrain.py</code>
context rasters	<code>scripts/prepare_change_context.py</code> , <code>scripts/prepare_hm_context.py</code>
context channels	<code>src/models/change_weights.py</code>
land, fold mask	<code>src/land.py</code> , <code>scripts/create_validity_mask.py</code> <code>--folds_only --fold_block_chips 4 --fold_min_valid_px 1</code>
location encoder	<code>src/locationencoder/</code>
trunk and heads	<code>src/models/convlstm.py</code> , <code>src/models/spatiotemporal_predictor.py</code>
quantile function, closed-form CRPS	<code>src/models/quantile_pwl.py</code>
loss and training loop	<code>src/models/lightning_module.py</code> , <code>scripts/train_lightning.py</code>
fold orchestration, stitching	<code>scripts/run_hindcast_folds.py</code> , <code>src/stitch.py</code>
scoring	<code>scripts/score_distributional_model.py</code>
products and verification	<code>scripts/export_products.py</code> , <code>scripts/verify_products.py</code>

References

- G. Amatulli, S. Domisch, M.-N. Tuanmu, B. Parmentier, A. Ranipeta, J. Malczyk and W. Jetz (2018). A suite of global, cross-scale topographic variables for environmental and biodiversity modeling. *Scientific Data* 5, 180040.
- J. Gasthaus, K. Benidis, Y. Wang, S. S. Rangapuram, D. Salinas, V. Flunkert and T. Januschowski (2019). Probabilistic forecasting with spline quantile function RNNs. *AISTATS*, 1901–1910.
- T. Gneiting and A. E. Raftery (2007). Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association* 102, 359–378.
- B. K. P. Horn (1981). Hill shading and the reflectance map. *Proceedings of the IEEE* 69, 14–47.
- D. N. Karger, O. Conrad, J. Böhner, T. Kawohl, H. Kreft, R. W. Soria-Auza, N. E. Zimmermann, H. P. Linder and M. Kessler (2017). Climatologies at high resolution for the earth’s land surface areas. *Scientific Data* 4, 170122.
- D. Murakami, T. Yoshida and Y. Yamagata (2021). Gridded GDP projections compatible with the five SSPs. *Frontiers in Built Environment* 7, 760306.
- J. R. Oakleaf, C. M. Kennedy, N. H. Wolff et al. (2024). Mapping global land conversion pressure to support conservation planning. *Scientific Data* 11, 830.

- Y. Park, D. Maddix, F.-X. Aubet, K. Kan, J. Gasthaus and Y. Wang (2022). Learning quantile functions without quantile crossing for distribution-free time series forecasting. *AISTATS*, 8127–8150.
- A. Paszke et al. (2019). PyTorch: an imperative style, high-performance deep learning library. *NeurIPS*.
- M. Rußwurm, K. Klemmer, E. Rolf, R. Zbinden and D. Tuia (2023). Geographic location encoding with spherical harmonics and sinusoidal representation networks. arXiv:2310.06743.
- X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-K. Wong and W.-C. Woo (2015). Convolutional LSTM network: a machine learning approach for precipitation nowcasting. *NeurIPS*.
- V. Sitzmann, J. N. P. Martel, A. W. Bergman, D. B. Lindell and G. Wetzstein (2020). Implicit neural representations with periodic activation functions. *NeurIPS*.
- D. M. Theobald, J. R. Oakleaf, G. Moncrieff, M. Voigt, J. Kiesecker and C. M. Kennedy (2025). Global extent and change in human modification of terrestrial ecosystems from 1990 to 2022. *Scientific Data* 12, 606.
- UNEP-WCMC and IUCN (2025). Protected Planet: the World Database on Protected Areas.
- R. Wen, K. Torkkola, B. Narayanaswamy and D. Madeka (2017). A multi-horizon quantile recurrent forecaster. NIPS Time Series Workshop, arXiv:1711.11053.
- H. Zhuang, X. Liu, B. Li, C. Wu, Y. Yan, L. Zeng and C. Zheng (2024). Mapping high-resolution global gridded population distribution from 1870 to 2100. *Science of the Total Environment* 955, 176867.